

Basic (your first real API call) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Simple Version

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a formal, professional assistant."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a sarcastic pirate."},
        {"role": "user", "content": "Tell me about your day."},
    ],
)
print(response.choices[0].message.content)
```

This works fine and shows the change clearly. It repeats the call structure twice, which is fine for a one-off script but gets repetitive fast if you compare more than two prompts.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI()

def ask(system_prompt: str, user_prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt},
        ],
    )
    return response.choices[0].message.content

def main() -> None:
    question = "Tell me about your day."

    formal_answer = ask("You are a formal, professional assistant.", question)
    print("Formal:", formal_answer)

    pirate_answer = ask("You are a sarcastic pirate.", question)
    print("Pirate:", pirate_answer)

if __name__ == "__main__":
    main()
```

What's different, and why it's better: the `ask()` function has full type hints and separates "how to call the API" from "which prompts to compare" — adding a third or fourth system prompt to compare is now one more `ask()` call, not a whole copy-pasted block. Wrapping the script's entry point in `main()` behind `if __name__ == "__main__":` also matches the pattern from Doc01's Build Task solution — this file could later be imported elsewhere without immediately running.

Which one should you use? The Simple version is fine to confirm the API call works at all. The Intermediate version's `ask()` function is what you actually want the moment you're comparing more than one prompt, or reusing this call anywhere else — which is exactly what happens next in this document's Build Task.